

Ex2 - THEORETICAL

Exam guide git.

2. a) $B^* b[i] = \{ \overset{b[i,0]}{\text{new } B \}, \overset{b[i,1]}{\text{new } D \} \}$;

$\rightarrow B() \Rightarrow$ "B33"

→ $D() \Rightarrow B() \Rightarrow \boxed{"B\{\}"}$

$\Rightarrow O() \Rightarrow \boxed{\text{"DSZ"}}$

$\Rightarrow$ Screen:

$$B \circ B \circ B \circ B \sim B() \sim D() \sim D()$$

b[0] → print(); ⇒ new B{} ^{virtual} ⇒ real type = B; pointer type = B*.

b[i] → print(); ⇒ new D{} ⇒ print() virtual in base class B
 ⇒ print() from class B automatically virtual.
 ⇒ real type = D

↳ `0::print()`

b) We can't create objects from abstract classes.

class Person $\rightarrow$ pure abstract

Person* p = new Person{}; ← ERROR!

constructor $E() = \boxed{\text{"{}{}{}"}}$

c) $\text{res} = 0$; $\text{DE}::m = 0$;

$res = fct2(-5) \Rightarrow -5 < 0 \Rightarrow \text{throw } \boxed{\{ \}} \Rightarrow$ nothing after throw is executed.
 $\Rightarrow$ nothing in "try" is executed after this throw;

$\Rightarrow$ catch:

$$DE \because m = 0.$$

$\Rightarrow$ after, catch, the thrown object (E?) gets destroyed.

destructor NE()

$${}^u NE()^u$$

⇒ Screen: E8YONE()

d) auto itBegin = d.begin()

```
auto itEnd = d.end()
```

itBegin++ (B)

itEnd -- (E)

while $\Rightarrow$ ~~B~~ A B A

⇒ Screen: BA

→ B A ←

→ D B A C ←

Diagram illustrating the effect of a CNOT gate on two qubits:

- Initial State (Left):** Two qubits are shown. The top qubit is in state $|0\rangle$ (green arrow pointing up) and the bottom qubit is in state $|0\rangle$ (green arrow pointing up). A curved arrow labeled $++$ indicates a CNOT gate operation.
- Intermediate State (Middle):** The top qubit is in state $|0\rangle$ (green arrow pointing up) and the bottom qubit is in state $|1\rangle$ (red arrow pointing up).
- Final State (Right):** The top qubit is in state $|1\rangle$ (red arrow pointing up) and the bottom qubit is in state $|0\rangle$ (red arrow pointing up). The word "AFTER" is written below the qubits, and a curved arrow labeled $--C$ indicates the CNOT gate operation.

"row 2" (the one with adder)

2. a) 10 Not empty
↓
return;

b) it = 4

v.insert(it, 7) ⇒ 7 before it(4).

it = v.begin() + 2 ⇒ it = 3.

*it = 11 (change value!)

copy-if ... ⇒ odd nr.

⇒ 1 11 7 5

1 | 2 | 3 | 4 | 5 |

↑
it.

1 | 2 | 3 | 7 | 4 | 5

↑
it

↓

1 | 2 | 11 | 7 | 4 | 5

c) B* b = new B{};

B* d = new D{};

d → f(); → Pointer type: B*

→ real object: D.

↳ virtual in the base class B ⇒ virtual here as well.

⇒ REAL OBJ. TYPE ⇒ D::f()

d → g(*b) → Pointer type: B*

→ real object: D.

↳ non-virtual

⇒ POINTER TYPE ⇒ B::g() ⇒ "B.g"

⇒ *b is a B object ⇒ constructor (copy) ⇒ "copy"

B b = *b ⇒ go out of scope ⇒ destructor ⇒ "NB"

Screen: B.f D.f copy B.g NB NB NB

~~delete b = B ⇒ destructor ⇒ NB~~

~~delete d ⇒ B* ⇒~~

delete b ⇒ real object: B ⇒ destructor ⇒ NB

delete d ⇒ real object: D ⇒ virtual destructor ⇒ D does not have one ⇒ use the base class (B) one ⇒ NB

A a1, a2; // a1 → 0;

// a2 → 0;

a1.set(8); // $\boxed{a1 \rightarrow 8}$

// $\boxed{a2 \rightarrow 0}$

A a3; // a3 → 0

a3 = a1; // ~~a3 → 0~~ automatically-generated assignment operator =.

⇒ $\boxed{a3.x = a1.x \rightarrow 8}$ (same 8)

A a4 = a1; // copy constructor

⇒ $\boxed{a4 = a1 = 8.}$ (another 8)

a1.set(5);

⇒ a1.x → 5
a3.x → 8
5

a1.get() ⇒ 5

a2.get() ⇒ 0

a3.get() ⇒ 5

a4.get() ⇒ 8

⇒ 5 0 5 8

! N A() { // a1.x == a3.x ⇒ double delete & dangling pointer.
delete x;
}

! {
 ✓ copy constructor,
 ✓ destructor
 ✓ copy assignment operator

! ~~SHALL~~

2018-1

2. a) 1 10 40

b) same as before (NB NB...)

c) answer to life 42

3+39=42

d) it.end() → after d.

it-- → at d

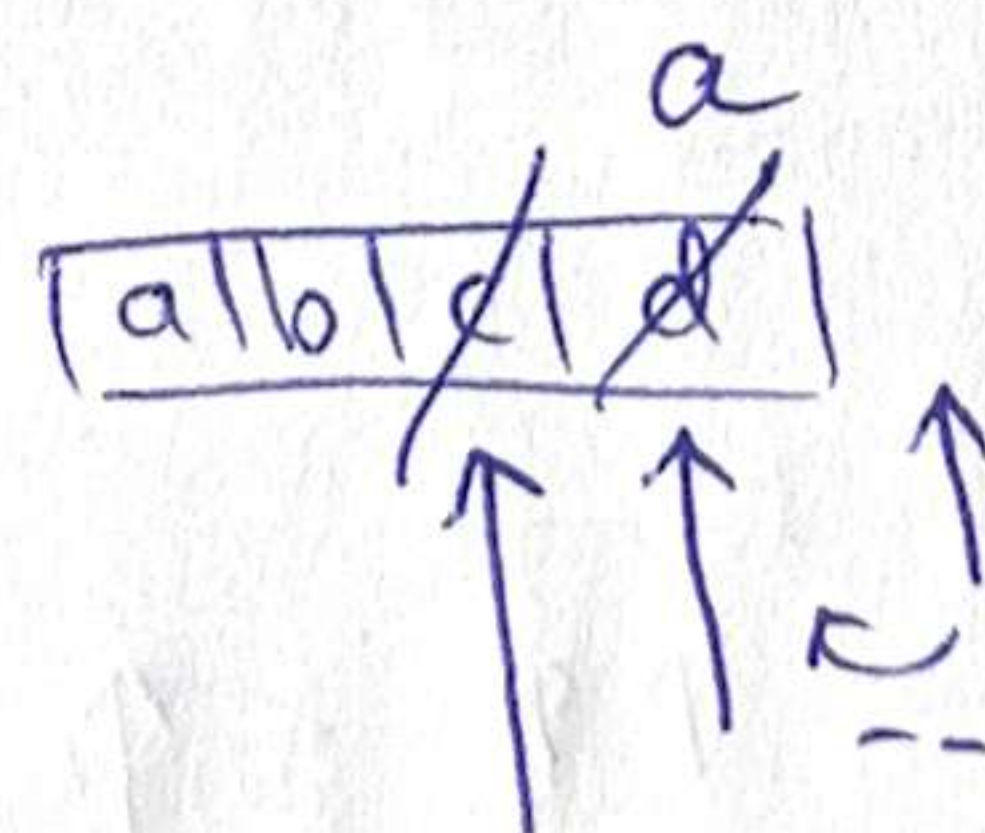
it-- → at c

ERROR!
uses invalid iterator!

! it = str.erase(it);
 ↑ str.insert(it, "b");

⇒ a b b a.

! correct ✓



2. a) $B^* b_1 = \text{new } B \{ \}$; $\rightarrow$ pointer: B^*
 $b_1 \rightarrow f()$ real: B

$B.f \ D \ B.f \ D.f \ \sim D \ \sim B \ \sim$!

$\hookrightarrow$ virtual in $B \Rightarrow$ real $\Rightarrow B::f() \Rightarrow \boxed{B.f}$

$B^* b_2 = \text{new } D \{ *b_1 \}$; $\rightarrow$ pointer: B^*
 $\downarrow$ real: B
 real: D

pointer: $B^* \Rightarrow$ constructor $\Rightarrow \boxed{D}$

$b_2 \rightarrow f()$

$\hookrightarrow b.f() \Rightarrow \boxed{B.f}$

$\hookrightarrow$ virtual in $B/D \Rightarrow$ real $\Rightarrow D::f() \Rightarrow \boxed{D.f}$

delete b_2 ;

$\hookrightarrow$ virtual destructor $\Rightarrow$ real: $D \Rightarrow \sim D() \Rightarrow \boxed{\sim D}$

delete b_1 ; $\Rightarrow$ destructor $\Rightarrow \sim B() \Rightarrow \boxed{\sim B}$

$\hookrightarrow$ virtual destructor $\Rightarrow$ real: $B \Rightarrow \sim B() \Rightarrow \boxed{\sim B}$

b) One Positive Negative

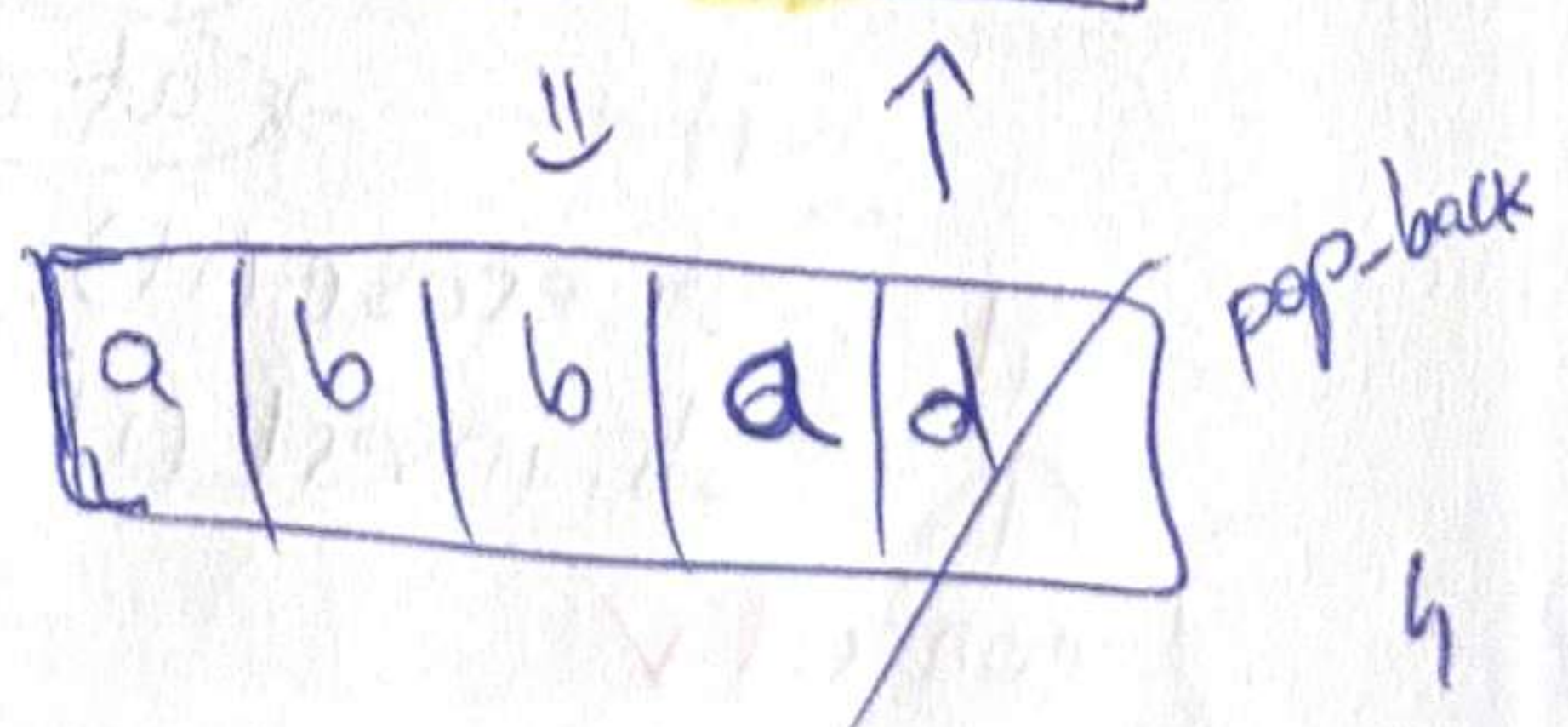
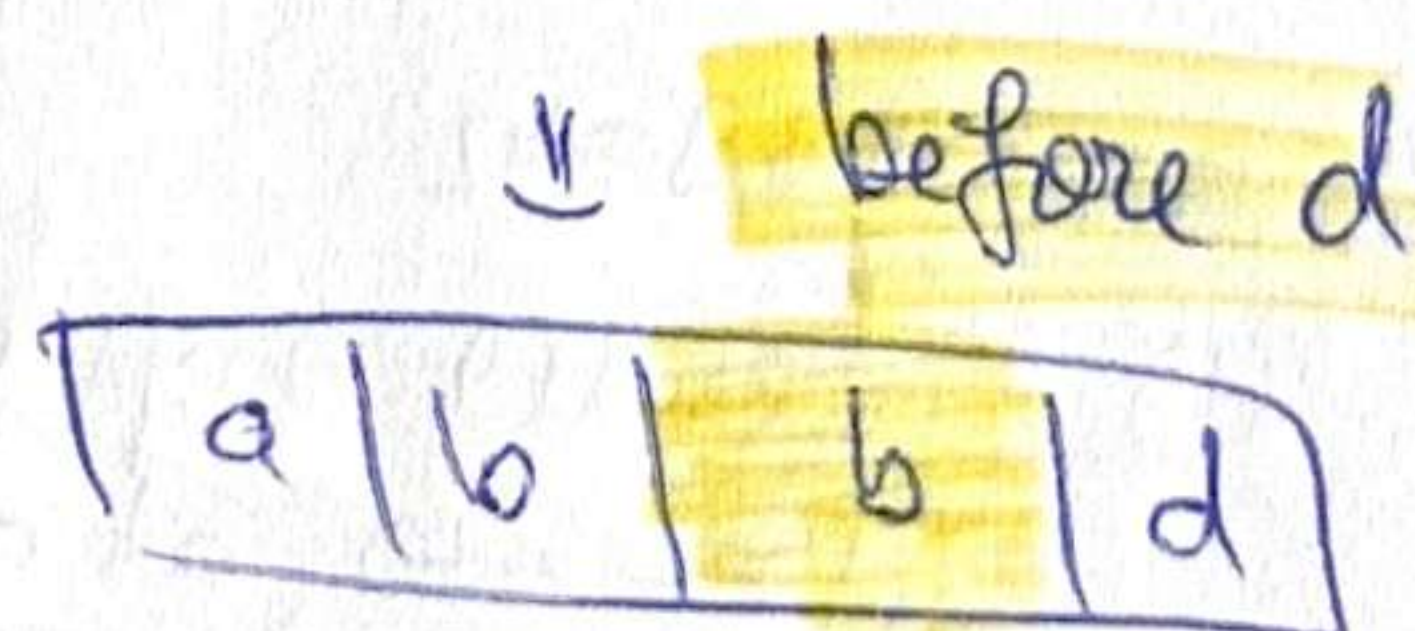
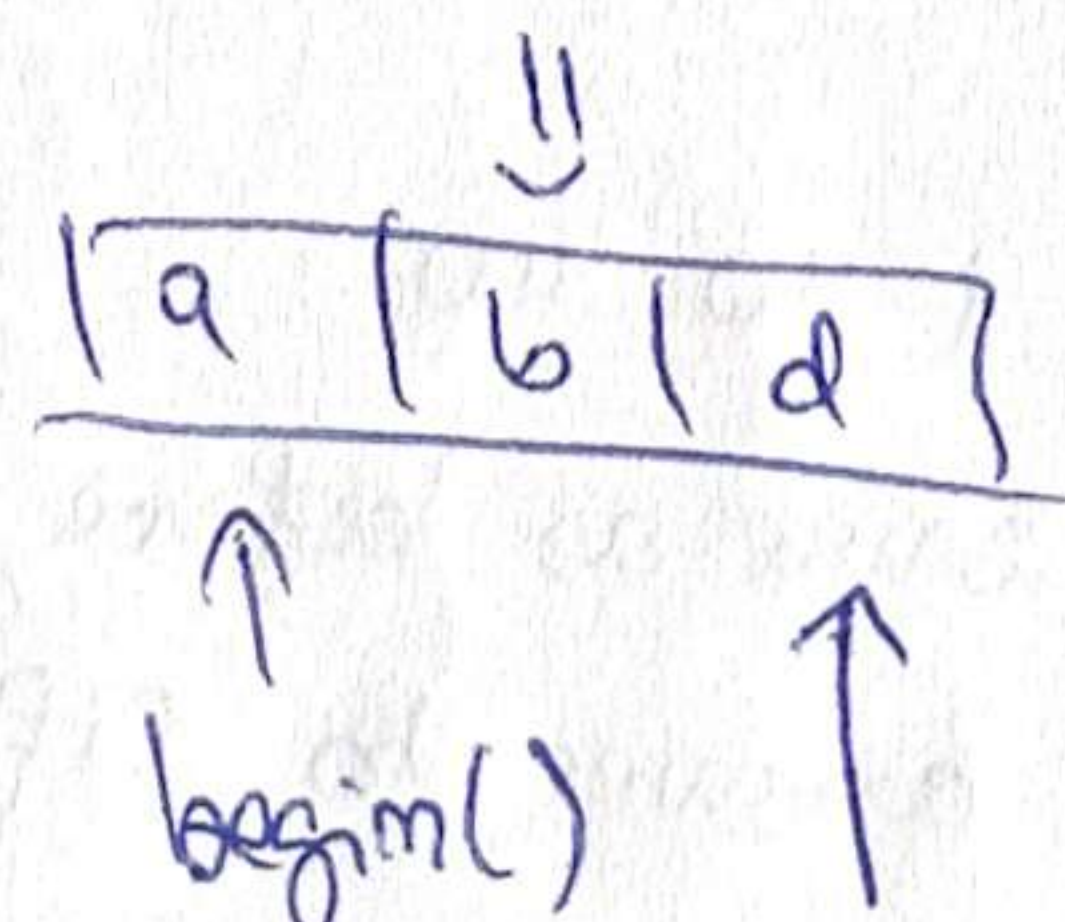
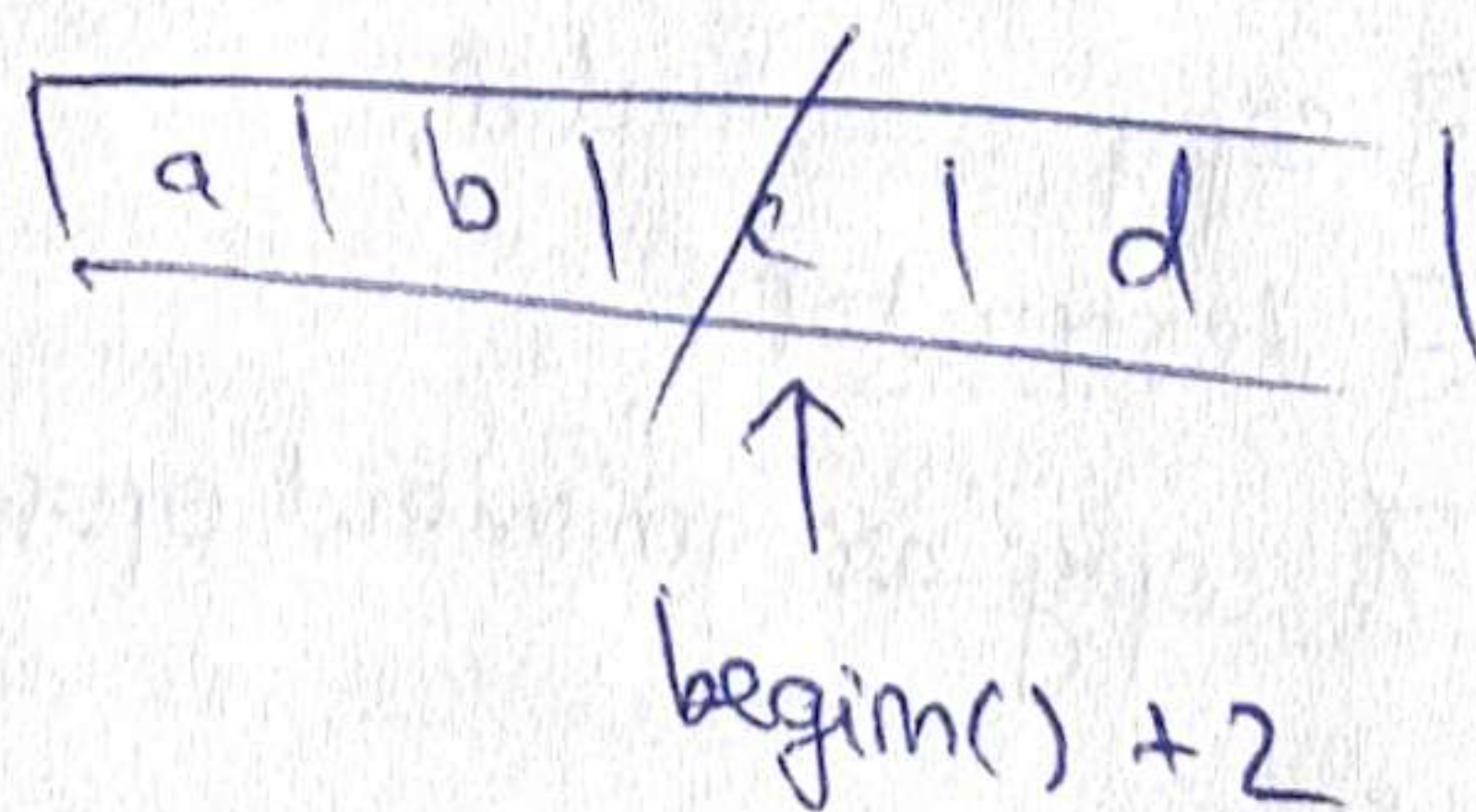
c) 10 10 10 10 10.5 5 -2 -2 Good luck! ~~1~~

$A \{2\} + A \{3\}$ can't be computed because A does not have operator $+$.

d) $\boxed{\text{end}() - 1} = \text{LAST ELEMENT!}$

pop-back() $\rightarrow$ delete LAST ELEMENT!

$\Rightarrow a b b a$



u3.jpg

2. a) 2 10 8 4

b) Start Exception1 Exception2

throw Ex3

↳ Exception2

↳ Ex1 → Exception1

↳ Ex2 → Exception2

except(-2)

↳ throw Ex1

↳ Exception1

↳ catch (Ex1 e)

↳ BY VALUE ⇒ copy constr.

⇒ "copy-ex1"

Screen:

Start Exception1 Exception2 Exception1 Copy-ex1

c) B* b1 = new B → pointer: B*
→ real: B

b1 → f()

↳ non-virtual ⇒ pointer: B* ⇒ B::f() ⇒ "B.f"

delete b1 -

B* b2 = new D1 → pointer: B*
→ real: D1

b2 → f()

↳ virtual ⇒ real: D1 ⇒ D1::f() ⇒ "D1.f"

delete b2 -

B* b3 = new D2 → p: B*
→ r: D2

b3 → f()

↳ ~~non~~ virtual ⇒ pointer: ~~B*~~ ⇒ ~~B.f~~
(inherits from B) real: D2 ⇒ D2::f() ⇒ "D2.f"

D1* d = new D2 → p: D1*
→ r: D2

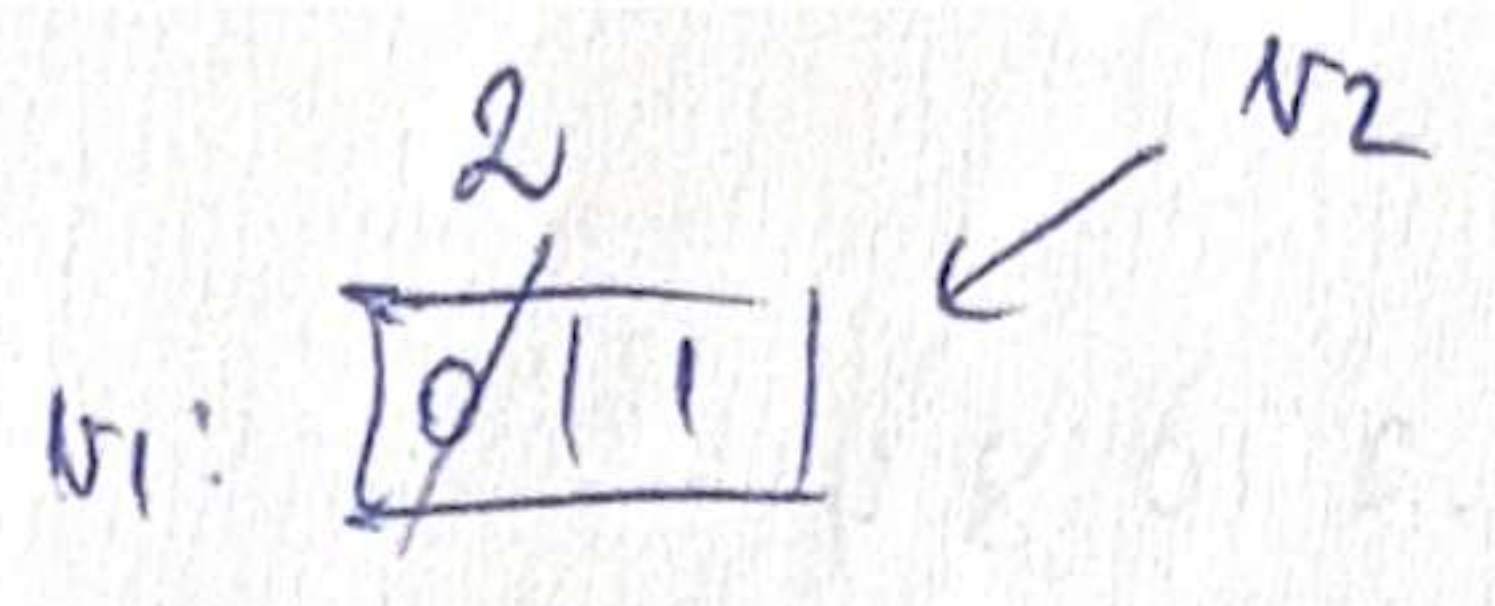
d → f()

↳ virtual ⇒ real: D2 ⇒ "D2.f"

Screen:

B.f D1.f D2.f D2.f

d) N1.add(0);
N1.add(1);



Vector N2 = N1; // no default copy constructor => shallow copy.

N1[0] = 2

Screen:

2 1 2 1

! goes out of scope => ~Vector() {
delete [] elems;
} ERROR
=> deletes twice ~ dangling pointer !!

- ✓ copy constructor
- ✓ destructor
- ✓ copy assignment operator,

"3.jpg"

TO REVIEW!

2. c) B* b = new B();
B* d = new D { b }
↳ constructor B() -> nothing
↳ constructor D(...) -> "0" ✓

* this

d -> f()
↳ virtual => real: D => D::f()
↳ D.f ✓
b -> f() => B.f ✓
deleted d;
↳ destructor of B -> NB

delete b
↳ its destructor -> NB

=> Screen: D D.f B.f NB NB
copy

A a1, a2; // a1 = 5 Screen: 2 2 10 5 10

a2 = 5

nrOfInstances = 2

A a3 = a1; // no default copy constructor => shallow copy (copy the pointer)

a1 -> 10 ← a3

a1.set(10)

! => goes out of scope => ~A() { delete x; }

=> ERROR (double delete)

"4.2 jpeg"

2. a) 0 1 Done

↓

return 0

except(-2)

↳ throw string {"4.2"}

↳ catch(string) => 1

b) { v.erase(it);
it = v.begin() + 3; } ✓ (valid)

=> 10 9 7 11 1

front_inserter(...) => inverse order of the odd numbers in the orig. vector

orig: 9 7 11 1

front_inserter -> 1 11 7 9

c) 0 - +

d) 0 2 0 10

=> final: 10 0 2 0

10	9	8	7	6	5
----	---	---	---	---	---

↑

11 1

10	9	7	8	6	5
----	---	---	---	---	---

↑ ↑

0	2	6	8	10
---	---	---	---	----

↑ ≠ 0

0	2	6	8	10
---	---	---	---	----

↑

2. a) $B^* b = \text{new } B\{\}$ $\rightarrow p: B^*, \text{real}: B$

$B^* d = \text{new } D\{\}$ $\rightarrow p: B^*, \text{real}: D$

$d \rightarrow f() \rightarrow B: f() \rightarrow \boxed{B.f}$
 $\hookrightarrow \boxed{D.f}$

$d \rightarrow g(*b)$

$\hookrightarrow$ copy constructor $\Rightarrow \boxed{\text{"copy"}}$
 $\hookrightarrow B^* \Rightarrow B::g(B b)$
 $(p) \hookrightarrow \boxed{B.g}$

$\hookrightarrow$ out of scope $\Rightarrow \boxed{NB}$ $\leftarrow$ destructor $NB()$

delete b; $\Rightarrow \boxed{NB}$
 delete d; $\Rightarrow \boxed{NB}$

$\Rightarrow$ Screen:

B.f D.f copy B.g NB NB NB.

b) A a1(2) // a1=2
 A a2(4) // a2=4

count=2 a4

a1.set(18) $\downarrow$ a3
 $\hookrightarrow a1 \rightarrow 18$

A a3; // a3=0, count=3

a3=a1; // ~~no copy constructor~~ $\Rightarrow$ shallow copy *assignment operator generated automatically*

A a4=a1; // shallow copy

a1.set(11)

~~2 2~~

Screen: 2
 11
 4
 11
 11
 3

916 - done already (99% similar)